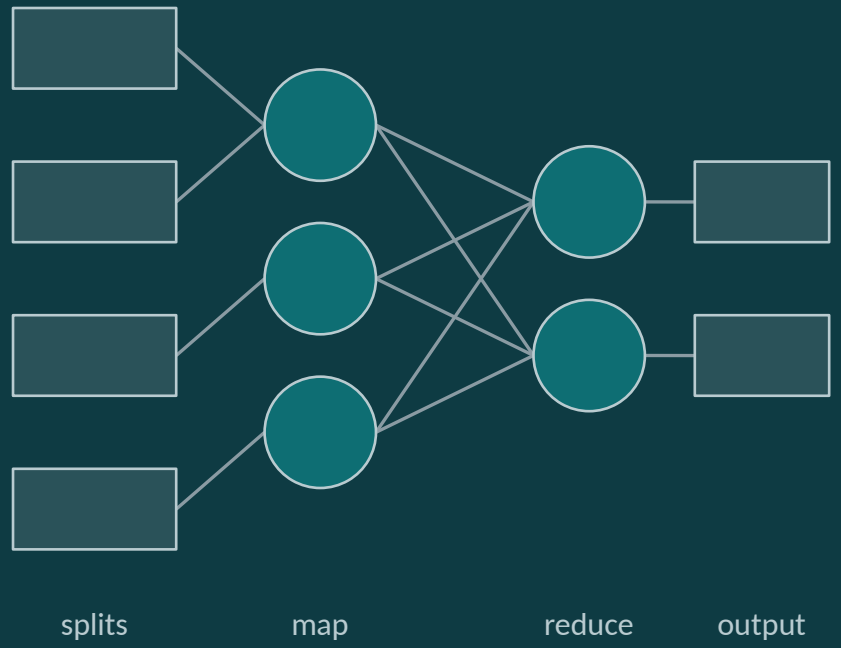
PAPER PRESENTATION · CS6650 MIDTERM MASTERY

MapReduce

Simplified Data Processing on Large Clusters

Jeffrey Dean and Sanjay Ghemawat · OSDI 2004

Focus: programming model, scalability and fault tolerance



M map tasks · R reduce tasks · all-to-all shuffle

Nikhil Shivakumar · Northeastern University · Fall 2026

Clusters offered capacity, but every job re-solved distribution

01 **Huge datasets on commodity machines**

Large clusters offer capacity, but individual workers fail routinely.

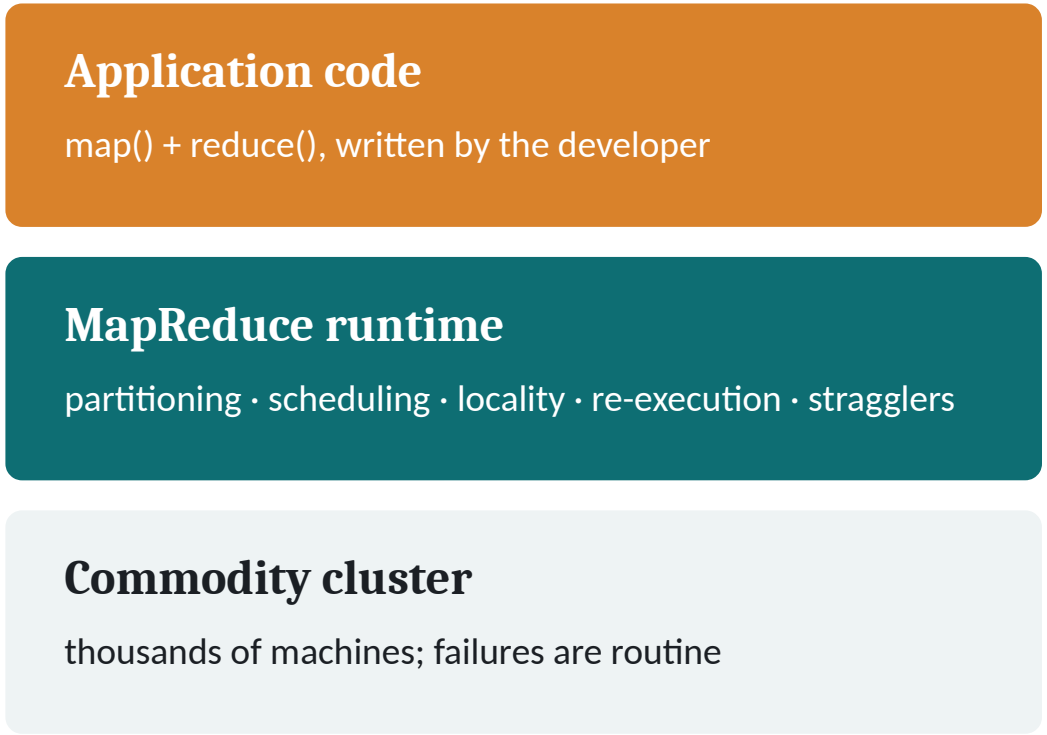
02 **Distribution was repeated engineering**

Each application rebuilt scheduling, data transfer and recovery.

03 **Separate logic from cluster management**

Developers write map and reduce; the runtime manages execution.

Fig. 1 Division of responsibility



A restricted model lets the runtime own distribution

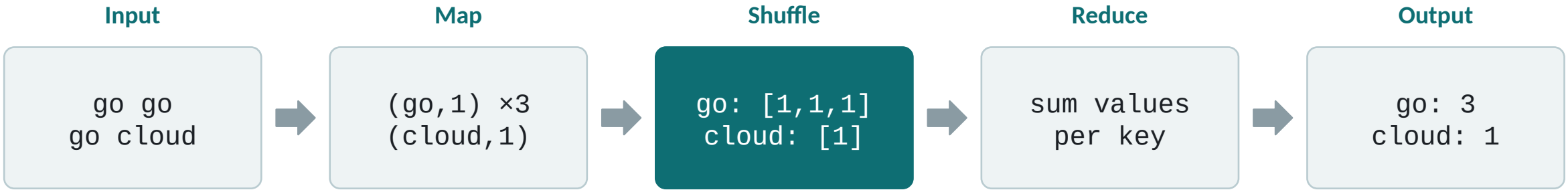


Fig. 2 Word count. The shuffle groups every value for a key at one of R reducers.

01

Restricted model

Map emits key–value pairs; hash(key) mod R sends each key to one reducer.

02

Locality

Run map tasks on or near the machines holding input replicas.

03

Re-execution

Recover from worker failures by rerunning their tasks.

04

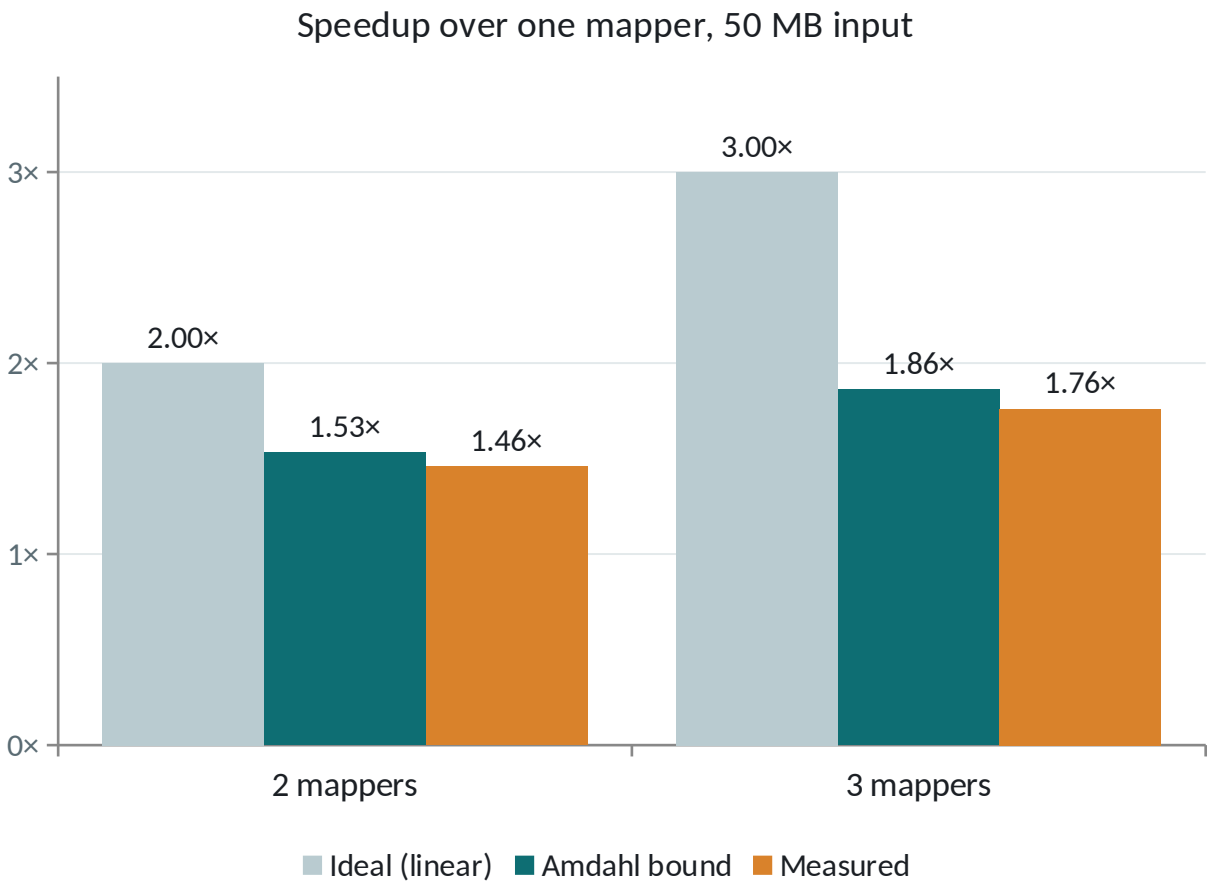
Backup tasks

Duplicate the last slow tasks; the first copy to finish wins.

The paper's mechanisms reappear across HW3–HW6

IN THE PAPER	IN MY ASSIGNMENTS	WHAT IT SHOWS
Keys partitioned across R reducers	**HW4** Splitter, 3 mappers and 1 reducer on Fargate; data passed through S3	**Maps parallelized; one reducer caps scale.** Partitioning across reducers removes it.
Tasks share no mutable state	**HW3 / HW5** One shared store, guarded by a sync.RWMutex	**Dataflow replaces locking:** each task writes its own output, and the shuffle moves data.
Master re-runs a failed worker's tasks	**HW6** ECS replaces failed tasks behind the ALB; auto scaling runs 2–4 tasks	**HW6 tasks held no unique state:** 0 failed requests with 2 of 3 stopped. MapReduce must recompute lost work.

HW4 speedup lands about 5% below the Amdahl bound



Where one mapper's time goes (50 MB)



$p = 4.11 / 5.92 = 0.69$

$S(3) \le 1 / (0.31 + 0.69/3) =$ ***1.86×*** ceiling $S(\infty) \le$ ***3.27×***

Why measured falls below the bound

Stragglers: mapper times varied ~10% on equal chunks

My serial splitter: it copied the whole file; the paper's byte-range splits avoid this. Estimated bound with S3 range reads: ~2.6×

Locality and cheap recovery let it scale; coordination caps it

WHAT LETS IT SCALE

Locality

§3.4

Map tasks run near input replicas, so most input is read locally and uses no network bandwidth.

Cheap recovery

§§3.6, 5.3

~1 TB sort with 200 of 1,746 workers killed finished only 5% slower. Without backup tasks, it ran 44% slower.

WHAT CAPS IT

Single master

§3.3

One master tracks every task. In the original implementation, its failure aborts the whole job.

Shuffle

§3.1

Every reducer fetches its partition from every map task: M × R transfers across the network.

Dean & Ghemawat, OSDI 2004 · M = map tasks, R = reduce tasks

Scale comes from independence; limits come from coordination

01

Independent tasks make batch work parallel

The runtime schedules work and reruns tasks lost to worker failures.

02

Coordination and data movement limit speedup

In HW4, my serial splitter was nearly half the 3-mapper runtime. The paper's design avoids that cost.

03

HW6 asks the same two questions

Which work is independent? What state survives a failure? HW6 tasks held no unique state, so stopping 2 of 3 cost zero requests.

Reference Dean, J. and Ghemawat, S. MapReduce: Simplified Data Processing on Large Clusters. *OSDI*, 2004.

Questions?